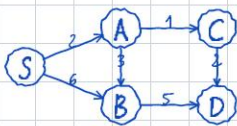


Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest paths from a source node to all other nodes in a graph with non-negative edge weights. It maintains a set of visited nodes and continuously expands the shortest-path frontier.



****Algorithm Steps (using a Min-Priority Queue):****

1. Initialize: Set distance to source = 0, all others = ∞ .
2. Push (0, source) into the PQ.
3. Pop the node u , with the minimum distance. If u is already visited, skip.
4. Relaxation: For each neighbor v of u , if $\text{dist}[u] + w(u, v) < \text{dist}[v]$:
 - Update $\text{dist}[v] = \text{dist}[u] + w(u, v)$.
 - Push $(\text{dist}[v], v)$ into the PQ.
5. Mark u as visited. Repeat until PQ is empty.

****Distance Table Updates:****

	Step	S	A	B	C	D
Step	S	A	B	C	D	
Init	0	∞	∞	∞	∞	∞
Pop S	0	2	6	∞	∞	
Pop A	0	2	5	3	∞	
Pop B	0	2	5	3	10	
Pop C	0	2	5	3	5	
Pop D	0	2	5	3	5	

Trace Through the Example:

- **Pop S:** Visit S. Relax $S \rightarrow A$ ($0+2 < \infty$) and $S \rightarrow B$ ($0+6 < \infty$). PQ has (2, A), (6, B).
- **Pop A:** Visit A. Relax $A \rightarrow B$ ($2+3=5 < 6$) and $A \rightarrow C$ ($2+1=3 < \infty$). PQ has (3, C), (5, B), (6, B).
- **Pop C:** Visit C. Relax $C \rightarrow D$ ($3+2=5 < \infty$). PQ has (5, B), (5, D), (6, B).
- **Pop B:** Visit B. Relax $B \rightarrow D$ ($5+5=10 > 5$). No update. PQ has (5, D), (6, B).
- **Pop D:** Visit D. PQ has (6, B).
- **Pop B:** B already visited, skip.

Final Shortest Paths from S:

- $S \rightarrow A$: 2
- $S \rightarrow B$: 5 (Path: $S \rightarrow A \rightarrow B$)
- $S \rightarrow C$: 3 (Path: $S \rightarrow A \rightarrow C$)
- $S \rightarrow D$: 5 (Path: $S \rightarrow A \rightarrow C \rightarrow D$)

Limitation:

Dijkstra's algorithm fails with negative edge weights. It assumes that once a node is popped from the PQ, its shortest path is permanently found. A negative weight could

later provide a shortcut to an already visited node, violating this greedy assumption.

Greedy + Relaxation = Optimal for non-negative weights